Bước 37 — GraphQL Directives

Sau Fragments (35) và Aliases (36), Bước 37 là Directives.

Câu hỏi quan trọng trước:

> Directive thuộc GraphQL hay Rails? Client có gửi directive trong request không?

Directive là cú pháp của GraphQL. Với hai directive chuẩn và thường dùng nhất là @include và @skip, client viết chúng trực tiếp trong GraphQL document gửi tới /graphql. Rails không cần tạo resolver riêng cho chúng.

1. Directive là gì?

Directive cho phép thay đổi cách GraphQL thực thi một phần của operation.

Cú pháp:

```
@directiveName(argument: value)
```

Hai directive quan trọng nhất:

```
@include(if: Boolean!)
@skip(if: Boolean!)
```

Hiểu đơn giản:

```
@include
→ l□y field n□u condition = true

@skip
→ b□ field n□u condition = true
```

2. @include

Ví dụ Project có:

```
id
name
description
tasks
```

Nhưng có lúc frontend chỉ cần:

```
id
name
```

Có lúc cần thêm:

```
tasks
```

Ta có thể viết:

```
query GetProject(
  $id: ID!
  $includeTasks: Boolean!
) {
  project(id: $id) {
    id
    name

    tasks @include(if: $includeTasks) {
      nodes {
        id
        title
      }
    }
  }
}
```

Điểm quan trọng:

```
@include(if: $includeTasks)
```

3. Variables cho @include

Client gửi:

```
{
  "id": "10",
```

```
  "includeTasks": true
}
```

Khi:

```
includeTasks = true
```

GraphQL lấy:

```
tasks
```

Response có thể là:

```
{
  "data": {
    "project": {
      "id": "10",
      "name": "CRM",
      "tasks": {
        "nodes": [
          {
            "id": "1",
            "title": "Build API"
          }
        ]
      }
    }
  }
}
```

4. Nếu includeTasks = false

Variables:

```
{
  "id": "10",
  "includeTasks": false
}
```

GraphQL bỏ:

```
tasks
```

Response:

```
{
  "data": {
    "project": {
      "id": "10",
      "name": "CRM"
    }
  }
}
```

Chú ý:

Không phải:

```
{
  "tasks": null
}
```

Mà field đó không xuất hiện trong response.

5. Client Axios hoàn chỉnh

```
import axios from "axios";

const query = `
  query GetProject(
    $id: ID!
    $includeTasks: Boolean!
  ) {
    project(id: $id) {
      id
      name

      tasks @include(if: $includeTasks) {
        nodes {
          id
          title
```

```
          status
        }
      }
    }
  }
`;

const response = await axios.post(
  "/graphql",
  {
    query,

    variables: {
      id: "10",
      includeTasks: true
    },

    operationName: "GetProject"
  },
  {
    withCredentials: true
  }
);
```

Directive nằm trong:

```
query
```

Variable:

```
includeTasks: true
```

nằm trong:

```
variables
```

6. Rails nhận gì?

GraphqlController vẫn hoàn toàn bình thường:

```
def execute
  result = MyAppSchema.execute(
    params[:query],
    variables: prepare_variables(params[:variables]),
    context: {
      current_user: current_user
    },
    operation_name: params[:operationName]
  )

  render json: result
end
```

Bạn không cần:

```
def include_tasks
end
```

Không cần:

```
class IncludeDirective
end
```

cho @include chuẩn này.

GraphQL engine xử lý nó.

7. @skip

@skip ngược lại với @include.

Ví dụ:

```
query GetProject(
  $id: ID!
  $skipDescription: Boolean!
) {
  project(id: $id) {
    id
    name
```

```
    description
      @skip(if: $skipDescription)
  }
}
```

Nếu:

```
{
  "id": "10",
  "skipDescription": true
}
```

thì:

```
description
→ bị b□
```

Response:

```
{
  "data": {
    "project": {
      "id": "10",
      "name": "CRM"
    }
  }
}
```

8. Nếu $skipDescription = false

Variables:

```
{
  "id": "10",
  "skipDescription": false
}
```

Response:

```
{
  "data": {
    "project": {
      "id": "10",
      "name": "CRM",
      "description": "Internal CRM project"
    }
  }
}
```

Công thức:

```
@skip(if: true)
→ b□

@skip(if: false)
→ l□y
```

Trong khi:

```
@include(if: true)
→ l□y

@include(if: false)
→ b□
```

9. Bảng ghi nhớ

```
| Directive | Condition | K□t qu□ |
|---|---:|---|
| `@include` | `true` | l□y field |
| `@include` | `false` | b□ field |
| `@skip` | `true` | b□ field |
| `@skip` | `false` | l□y field |
```

Đây là phần cần nhớ nhất.

10. Directive + Fragment

Đây là chỗ Bước 35 + Bước 37 kết hợp.

Fragment:

```
fragment TaskFields on Task {
  id
  title
  status
  priority
}
```

Query:

```
query GetProject(
  $id: ID!
  $includeTasks: Boolean!
) {
  project(id: $id) {
    id
    name

    tasks @include(if: $includeTasks) {
      nodes {
        ...TaskFields
      }
    }
  }
}
```

Ở đây:

```
...TaskFields
→ Fragment

@include
→ Directive
```

11. Directive có thể đặt trên Fragment Spread

Không chỉ field.

Bạn có thể:

```
query GetProject(
  $id: ID!
  $includeDetails: Boolean!
) {
  project(id: $id) {
    id
    name

    ...ProjectDetails
      @include(if: $includeDetails)
  }
}

fragment ProjectDetails on Project {
  description
  status
  createdAt
  updatedAt
}
```

Nếu:

```
{
  "id": "10",
  "includeDetails": false
}
```

thì toàn bộ:

```
...ProjectDetails
```

bị bỏ.

12. Directive + Alias + Fragment

Bây giờ ghép Bước 35 + 36 + 37:

```
fragment TaskSummaryFields on Task {
  id
  title
  status
  priority
}

query GetTaskBoard(
  $projectId: ID!
  $includeDone: Boolean!
) {
  todoTasks: tasks(
    projectId: $projectId
    filter: {
      status: [TODO]
    }
    first: 10
  ) {
    nodes {
      ...TaskSummaryFields
    }
  }

  doneTasks: tasks(
    projectId: $projectId
    filter: {
      status: [DONE]
    }
    first: 10
  ) @include(if: $includeDone) {
    nodes {
      ...TaskSummaryFields
    }
  }
}
```

Ở đây:

```
GetTaskBoard
→ Operation Name

$projectId
$includeDone
→ Variables

todoTasks
doneTasks
→ Aliases

...TaskSummaryFields
→ Fragment

@include
→ Directive
```

Đây là một GraphQL request thực tế.

13. Axios cho request trên

```
const response = await axios.post(
  "/graphql",
  {
    query: `
      fragment TaskSummaryFields on Task {
        id
        title
        status
        priority
      }

      query GetTaskBoard(
        $projectId: ID!
        $includeDone: Boolean!
      ) {
        todoTasks: tasks(
          projectId: $projectId
          filter: {
            status: [TODO]
```

```
          }
          first: 10
        ) {
          nodes {
            ...TaskSummaryFields
          }
        }

        doneTasks: tasks(
          projectId: $projectId
          filter: {
            status: [DONE]
          }
          first: 10
        ) @include(if: $includeDone) {
          nodes {
            ...TaskSummaryFields
          }
        }
      }
    `,

    variables: {
      projectId: "10",
      includeDone: true
    },

    operationName: "GetTaskBoard"
  },
  {
    withCredentials: true
  }
);
```

14. Tại sao không xử lý bằng JavaScript?

Bạn có thể nghĩ:

```
if (includeTasks) {
  // query có tasks
} else {
  // query không có tasks
}
```

Sau đó tạo hai query:

```
GET_PROJECT
```

và:

```
GET_PROJECT_WITH_TASKS
```

Có thể làm, nhưng khi khác biệt chỉ là một vài field thì directive thường sạch hơn:

```
tasks @include(if: $includeTasks)
```

Một operation:

```
GetProject
```

một variable:

```
includeTasks
```

dễ quản lý hơn.

15. Directive có giúp giảm dữ liệu response không?

Có.

Nếu:

```
tasks @include(if: false)
```

thì tasks không được đưa vào result.

Điều này có thể giúp:

```
response nh□ hơn
ít field c□n resolve hơn
ít công việc không c□n thi□t
```

Nhưng phải hiểu chính xác:

> Directive không phải database optimization tool.

16. Directive có giảm SQL không?

Có thể gián tiếp tránh một resolver và SQL tương ứng nếu field bị bỏ.

Ví dụ:

```
tasks @include(if: false)
```

GraphQL không cần resolve tasks.

Nếu tasks resolver bình thường chạy:

```
project.tasks
```

thì query đó có thể không chạy.

Nhưng không nên kết luận:

```
directive
=
SQL optimizer
```

Không đúng.

N+1 vẫn phải xử lý bằng:

```
Dataloader
preload
includes
batch loading
```

17. Directive có dùng cho Authorization không?

Không.

Đây là lỗi rất nguy hiểm.

Không làm:

```
adminData
  @include(if: $isAdmin)
```

rồi tin rằng:

```
{
  "isAdmin": false
}
```

sẽ bảo mật dữ liệu.

Client hoàn toàn có thể gửi:

```
{
  "isAdmin": true
}
```

Do đó:

```
@include
@skip
```

không phải authorization.

18. Authorization phải ở Rails

Ví dụ:

```
def resolve(project_id:, **)
  project =
    Project.find(project_id)

  authorize project, :show?

  project
end
```

Hoặc field authorization phù hợp với kiến trúc của bạn.

Nguồn tin cậy:

```
context[:current_user]
```

đến từ:

```
current_user
```

của Devise.

Không phải:

```
$isAdmin
```

do client gửi.

19. Sai và đúng về security

Sai

```
query GetProject(
  $isAdmin: Boolean!
) {
  project(id: "10") {
    secretData
      @include(if: $isAdmin)
  }
}
```

Client:

```
{
  "isAdmin": true
}
```

Client tự quyết định được boolean.

Đó không phải security.

Đúng

Rails:

```
def secret_data
  user =
    context[:current_user]

  raise GraphQL::ExecutionError,
        "Forbidden" unless user.admin?

  object.secret_data
end
```

Directive chỉ quyết định:

```
client có mu□n request field hay không
```

Policy quyết định:

```
client có quy□n l□y field hay không
```

20. Directive và React UI

Giả sử UI có checkbox:

```
<label>
  <input
    type="checkbox"
    checked={includeDone}
    onChange={(event) =>
      setIncludeDone(event.target.checked)
    }
  />

  Show done tasks
</label>
```

State:

```
const [
  includeDone,
  setIncludeDone
```

```
] = useState(false);
```

Request:

```
variables: {
  projectId: "10",
  includeDone
}
```

GraphQL:

```
doneTasks
  @include(if: $includeDone)
```

Đây là use case rất rõ của directive.

21. Không cần build query string động

Không nên:

```
let fields = `
  id
  name
`;

if (includeTasks) {
  fields += `
    tasks {
      nodes {
        id
      }
    }
  `;
}
```

rồi:

```
const query = `
  query {
    project {
      ${fields}
    }
  }
`;
```

Cách này khó:

```
maintain
test
type check
persisted query
operation tracking
```

Tốt hơn:

```
tasks
  @include(if: $includeTasks)
```

22. Directive với Boolean default value

Bạn có thể khai báo:

```
query GetProject(
  $id: ID!
  $includeTasks: Boolean! = false
) {
  project(id: $id) {
    id
    name

    tasks @include(if: $includeTasks) {
      nodes {
        id
        title
      }
    }
  }
}
```

Nếu client không gửi:

```
{
  "includeTasks": ...
}
```

thì:

```
includeTasks = false
```

23. Axios khi dùng default value

Client chỉ cần:

```
variables: {
  id: "10"
}
```

Không cần:

```
includeTasks: false
```

vì operation đã có:

```
$includeTasks: Boolean! = false
```

24. Có Custom Directive không?

Có.

Ngoài:

```
@include
@skip
```

GraphQL implementation có thể hỗ trợ custom directives.

Ví dụ về mặt ý tưởng:

```
@auth
@deprecated
@upper
@cache
```

Nhưng phải phân biệt.

@include và @skip:

```
GraphQL standard executable directives
```

Custom directive:

```
do application/server tự định nghĩa
```

25. @deprecated cũng là Directive

Ví dụ schema:

```
type Project {
  oldName: String
    @deprecated(
      reason: "Use name instead"
    )

  name: String!
}
```

Trong graphql-ruby, bạn thường khai báo deprecation qua API của field, ví dụ:

```
field :old_name,
      String,
      null: true,
      deprecation_reason: "Use name instead"
```

Đây liên quan trực tiếp tới:

Bước 43 — Schema Evolution & Deprecation.

26. Executable Directive vs Schema Directive

Nên phân biệt hai ý.

Executable

Được client sử dụng trong operation:

```
@include
@skip
```

Ví dụ:

```
tasks
  @include(if: $includeTasks)
```

Schema-related directive

Mô tả/hành xử ở schema:

```
@deprecated
```

Ví dụ:

```
oldName
  @deprecated(reason: "Use name")
```

Cho nên không phải mọi directive đều có cùng mục đích.

27. Directive + Pagination

Ví dụ Project detail có thể optional load Tasks:

```
query GetProject(
  $id: ID!
  $includeTasks: Boolean!
  $first: Int!
  $after: String
) {
  project(id: $id) {
    id
    name

    tasks(
      first: $first
      after: $after
    ) @include(if: $includeTasks) {
      nodes {
        id
        title
      }

      pageInfo {
        hasNextPage
        endCursor
      }
    }
  }
}
```

Ở đây:

```
$first / $after
→ Pagination

$includeTasks
→ Directive condition
```

28. Directive + Filter + Sort + Pagination

Bây giờ ghép các bước trước:

```
query GetProjectTasks(
  $projectId: ID!
  $filter: TaskFilterInput
  $sortBy: TaskSortFieldEnum
  $sortDirection: SortDirectionEnum
  $first: Int!
  $after: String
  $includeTasks: Boolean!
) {
  project(id: $projectId) {
    id
    name

    tasks(
```

```
      filter: $filter
      sortBy: $sortBy
      sortDirection: $sortDirection
      first: $first
      after: $after
    ) @include(if: $includeTasks) {
      nodes {
        ...TaskSummaryFields
      }

      pageInfo {
        hasNextPage
        endCursor
      }
    }
  }
}
```

Đây đã là GraphQL query khá sát production.

29. Client variables

```
const variables = {
  projectId: "10",

  filter: {
    status: [
      "TODO",
      "IN_PROGRESS"
    ],

    priority: [
      "HIGH"
    ]
  },

  sortBy: "DUE_DATE",
  sortDirection: "ASC",

  first: 20,
  after: null,

  includeTasks: true
};
```

30. Fragment + Alias + Directive + Variables

Một ví dụ tổng hợp:

```
fragment TaskSummaryFields on Task {
  id
  title
  status
  priority
  dueDate
}

query GetProjectBoard(
  $projectId: ID!
  $includeTodo: Boolean!
  $includeDone: Boolean!
) {
  project(id: $projectId) {
    id
    name
  }

  todoTasks: tasks(
    projectId: $projectId
    filter: {
      status: [TODO]
    }
    first: 10
  ) @include(if: $includeTodo) {
    nodes {
      ...TaskSummaryFields
    }
```

```
  }

  doneTasks: tasks(
    projectId: $projectId
    filter: {
      status: [DONE]
    }
    first: 10
  ) @include(if: $includeDone) {
    nodes {
      ...TaskSummaryFields
    }
  }
}
```

Variables:

```
{
  "projectId": "10",
  "includeTodo": true,
  "includeDone": false
}
```

Response:

```
{
  "data": {
    "project": {
      "id": "10",
      "name": "CRM"
    },

    "todoTasks": {
      "nodes": [
        {
          "id": "1",
          "title": "GraphQL API",
          "status": "TODO",
          "priority": "HIGH",
          "dueDate": "2026-08-25"
        }
      ]
    }
  }
}
```

doneTasks không xuất hiện vì:

```
includeDone = false
```

31. Sơ đồ Bước 37

```
CLIENT
│
├── Operation
│   GetProjectBoard
│
├── Variables
│   projectId
│   includeTodo
│   includeDone
│
├── Aliases
│   todoTasks
│   doneTasks
│
├── Fragment
│   TaskSummaryFields
│
└── Directives
    @include
        │
        ▼
GraphQL Document
        │
        ▼
POST /graphql
        │
        ▼
```

```
GraphQL Engine
        │
        ├── validate variables
        ├── validate fragments
        ├── evaluate directives
        └── build execution
        │
        ▼
Rails Resolver
        │
        ▼
Policy / Service
        │
        ▼
Database
        │
        ▼
Response
```

32. Bốn khái niệm rất dễ nhầm

```
Variable
→ dữ liệu động

Fragment
→ reuse fields

Alias
→ đ□i response key

Directive
→ đi□u khi□n có thực thi/include
  selection hay không
```

Ví dụ:

```
query GetBoard(
  $projectId: ID!
  $showDone: Boolean!
) {
  doneTasks: tasks(
    projectId: $projectId
  ) @include(if: $showDone) {
    nodes {
      ...TaskFields
    }
  }
}
```

Phân tích:

```
GetBoard
→ Operation Name

$projectId
$showDone
→ Variables

doneTasks
→ Alias

tasks
→ field thật trong Schema

@include
→ Directive

...TaskFields
→ Fragment
```

Ghi nhớ Bước 37

```
Directive
→ thuộc GraphQL

Được vi□t □ đâu?
→ GraphQL document c□a client

Có nằm trong Axios request?
```

```
→ có, bên trong query string

Rails có c□n resolver riêng
cho @include / @skip?
→ không

@include(if: true)
→ l□y

@include(if: false)
→ b□

@skip(if: true)
→ b□

@skip(if: false)
→ l□y

Có dùng đ□ authorization?
→ KHÔNG

Có th□ gi□m field được resolve?
→ có

Có ph□i SQL optimizer?
→ không
```

Điểm quan trọng nhất của Bước 37 là:

```
Variable
       ↓
$includeTasks
       ↓
Directive
       ↓
@include(if: $includeTasks)
       ↓
GraphQL quy□t định
có execute selection đó hay không
```

Và từ Bước 33 → 37, bạn đã có chuỗi kiến thức client GraphQL:

```
33 Variables
     ↓
34 Operation Name
     ↓
35 Fragments
     ↓
36 Aliases
     ↓
37 Directives
```

Tất cả ghép lại trong GraphQL document mà frontend gửi đến Rails /graphql.